

ECOLE MAROCAINE DES SCIENCES DE L'INGENIEUR

Casablanca

Membre de HONORIS UNITED UNIVERSITIES

PROJET DE FIN DE MODULE Deep Learning

Deep Learning XAI

Conception, Implementation et Analyse de Modeles
de Deep Learning avec Explainable AI
pour les Donnees Tabulaires, Images et Sequentielles

Realise par :
YAHYA BARK

Filiere :
Informatique

Annee :
3eme Annee

Encadre par :
[Nom du Professeur]

Departement :
Informatique

Module :
Deep Learning

Annee Universitaire : **2025-2026**

Ce projet constitue l'evaluation finale du module. Il s'agit d'un travail individuel.

Dedicaces

Je dedie ce travail a mes parents qui m'ont toujours soutenu et encourage dans mon parcours academique. Leur amour et leurs sacrifices ont ete la source de ma motivation et de ma perseverance.

Je remercie egalement mes enseignants pour leur devouement et leurs connaissances transmises avec passion.

Remerciements

Je tiens à exprimer ma gratitude à :

- **Monsieur [Nom du Professeur]**, mon encadrant, pour son accompagnement, ses conseils avisés et sa disponibilité tout au long de ce projet.
- **L'EMSI Casablanca** pour la qualité de l'enseignement et les ressources mises à disposition.
- Mes camarades de promotion pour les échanges enrichissants et l'entraide.
- Toute l'équipe pédagogique du département Informatique.

Resume

Ce projet explore l'implementation et l'analyse de differentes architectures de deep learning appliquees a trois types de donnees : tabulaires, images et sequentielles. L'objectif est de demontrer la capacite d'adaptation des reseaux de neurones profonds aux caracteristiques specifiques de chaque type de donnee, tout en integrant des techniques d'Explainable AI (XAI) pour l'interpretation des modeles.

Les contributions principales incluent :

- Implementation d'un MLP pour la classification de donnees tabulaires (Wine Quality) avec analyse SHAP
- Conception d'un CNN et d'un modele hybride CNN-LSTM pour la classification d'images (CIFAR-10)
- Modelisation de sequences avec RNN, LSTM, GRU et implementation d'un systeme Seq2Seq avec mecanisme d'attention
- Fusion multi-agents (MLP + CNN + RNN) via des strategies de vote et de meta-learning
- Etude d'ablation pour quantifier l'impact de chaque composante
- Dashboard XAI global combinant SHAP, Grad-CAM et visualisation de l'attention

Mots-cles : Deep Learning, PyTorch, MLP, CNN, RNN, LSTM, GRU, Seq2Seq, Attention, XAI, SHAP, Grad-CAM, Ablation Study, Agents Collaboratifs.

Table des matières

Dedicaces	1
Remerciements	2
Resume	3
1 Introduction Generale	7
1.1 Contexte et Motivation	7
1.2 Problematique	7
1.3 Objectifs du Projet	7
1.4 Organisation du Rapport	8

2	Partie I : Perceptron Multi-Couche (MLP)	9
2.1	Introduction Theorique	9
2.1.1	Fondements du Perceptron Multi-Couche	9
2.1.2	Fonction d'Activation ReLU	9
2.1.3	PyTorch : nn.Module vs nn.Sequential	10
2.1.4	Initialisation des Poids	11
2.2	Implementation	11
2.2.1	Dataset : Wine Quality	11
2.2.2	Preparation des Donnees	12
2.2.3	Architecture du MLP	12
2.2.4	Entrainement avec Early Stopping	13
2.3	Etude Experimentale	14
2.3.1	Resultats d'Entrainement	14
2.3.2	Visualisation des Courbes d'Entrainement	15
2.3.3	Matrice de Confusion	16
2.4	Explainable AI : SHAP	16
2.4.1	Principe de SHAP	16
2.4.2	Resultats SHAP	17
2.5	Ablation Study	18
2.6	Question de Synthese Partie I	19
3	Partie II : Reseaux de Neurones Convolutifs (CNN)	21
3.1	Introduction Theorique	21
3.1.1	Pourquoi les CNN pour les Images ?	21
3.1.2	Operation de Convolution 2D	21
3.1.3	Pooling et Sous-echantillonnage	22
3.2	Implementation du CNN Basique	22
3.2.1	Architecture LeNet-like	22
3.2.2	Calculs Dimensionnels	23
3.3	Modele Hybride CNN-LSTM	23
3.3.1	Motivation	23
3.3.2	Implementation	23
3.4	XAI : Grad-CAM	24
3.4.1	Principe de Grad-CAM	24
3.4.2	Resultats	25
3.5	Analyse Comparative	25
3.6	Question de Synthese Partie II	25
4	Partie III : Modeles Recurrents et Seq2Seq	27
4.1	Introduction Theorique	27
4.1.1	Modelisation Probabiliste des Sequences	27
4.1.2	Perplexite	27
4.1.3	Le Probleme des RNN Simples	27
4.1.4	LSTM : Long Short-Term Memory	28
4.1.5	GRU : Gated Recurrent Unit	28
4.2	Implementation	29
4.2.1	Generation de Texte	29
4.2.2	Seq2Seq avec Attention	29
4.3	Resultats	30

4.4	Question de Synthèse Partie III	30
5	Partie IV : Agents Collaboratifs	31
5.1	Architecture Multi-Agents	31
5.2	Méthodes de Fusion	31
6	Partie V : Ablation Study Globale	32
7	Partie VI : Dashboard XAI Global	34
8	Conclusion Générale	35
8.1	Synthèse	35
8.2	Perspectives	35

Table des figures

2.1	Architecture du MLP implémenté	13
2.2	Courbes de loss et d'accuracy pendant l'entraînement	15
2.3	Matrice de confusion sur le test set	16
2.4	SHAP Summary Plot pour la classe 0	17
2.5	Importance globale des features selon SHAP	18
2.6	Impact de l'ablation de chaque feature sur l'accuracy	19
3.1	Architecture Hybride CNN-LSTM	23
3.2	Grad-CAM : Zones importantes pour la classification	25
4.1	Structure de la cellule LSTM	28
4.2	Matrice d'attention : Alignement source-cible	30
5.1	Système Multi-Agents Collaboratifs	31
6.1	Impact global de l'ablation	32
7.1	Dashboard XAI Global unifié	34

Liste des tableaux

2.1	Comparaison des stratégies d'initialisation	11
2.2	Performances du MLP sur le dataset Wine	14
2.3	Résultats de l'ablation study	19

3.1	Impact des hyperparametres de convolution	22
3.2	Evolution des dimensions a travers le reseau	23
3.3	Comparaison MLP vs CNN sur CIFAR-10	25
4.1	Comparaison RNN / LSTM / GRU	29
5.1	Comparaison des methodes de fusion	31
6.1	Resultats complets de l'ablation study	33

Chapitre 1

Introduction Generale

1.1 Contexte et Motivation

L'intelligence artificielle connaît une révolution sans précédent, portée en grande partie par les avancées du deep learning. Les réseaux de neurones profonds ont démontré leur capacité à résoudre des problèmes complexes dans des domaines aussi variés que la vision par ordinateur, le traitement du langage naturel, la reconnaissance vocale et l'analyse de données tabulaires.

Cependant, ces modèles sont souvent perçus comme des *boîtes noires*, rendant difficile l'interprétation de leurs décisions. C'est dans ce contexte que l'Explainable AI (XAI) émerge comme un domaine crucial, visant à rendre les modèles de deep learning transparents et interprétables.

1.2 Problematique

Comment le deep learning adapte-t-il ses architectures aux différentes structures de données (tabulaires, images, séquentielles), et comment peut-on expliquer et justifier les décisions de ces modèles complexes ?

1.3 Objectifs du Projet

Ce projet vise à répondre à cette problématique à travers les objectifs suivants :

1. **Comprehension theorique** : Maîtriser les fondements mathématiques et algorithmiques des architectures MLP, CNN et RNN/LSTM/GRU.
2. **Maîtrise technique** : Implémenter ces modèles sous PyTorch avec une architecture modulaire et réutilisable.
3. **Innovation** : Concevoir un modèle hybride CNN-LSTM et un système multi-agents collaboratifs.
4. **Explicabilité** : Intégrer des techniques XAI (SHAP, Grad-CAM, Attention) pour interpréter les prédictions.
5. **Analyse critique** : Conduire des études ablation pour quantifier l'impact de chaque composante.

1.4 Organisation du Rapport

Ce rapport est structure en six parties principales, suivies d'une conclusion generale et d'une bibliographie :

Partie I

Perceptron Multi-Couche (MLP) pour les donnees tabulaires

Partie II

Reseaux de Neurones Convolutifs (CNN) et modele hybride CNN-LSTM pour les images

Partie III

Modeles recurrents (RNN, LSTM, GRU) et Seq2Seq avec attention pour les sequences

Partie IV

Agents collaboratifs et fusion multi-modeles

Partie V

Etude d'ablation globale

Partie VI

Dashboard XAI global et synthese des techniques d'explicabilite

Chapitre 2

Partie I : Perceptron Multi-Couche (MLP)

2.1 Introduction Theorique

2.1.1 Fondements du Perceptron Multi-Couche

Le Perceptron Multi-Couche (MLP) constitue l'architecture fondamentale des reseaux de neurones profonds. Introduit dans les annees 1980 et popularise par l'algorithme de retropropagation, le MLP est capable d'approximer n'importe quelle fonction continue, ce qui en fait un outil universel pour la classification et la regression.

Mathematiquement, un MLP a L couches est defini par la composition de fonctions :

$$f(x) = f^{[L]} \circ f^{[L-1]} \circ \dots \circ f^{[1]}(x) \quad (2.1)$$

ou chaque couche l applique une transformation affine suivie d'une activation non-lineaire :

$$z^{[l]} = W^{[l]} \cdot a^{[l-1]} + b^{[l]} \quad (2.2)$$

$$a^{[l]} = \sigma(z^{[l]}) \quad (2.3)$$

Dans ces equations :

- $W^{[l]} \in \mathbb{R}^{n_l \times n_{l-1}}$ est la matrice de poids
- $b^{[l]} \in \mathbb{R}^{n_l}$ est le vecteur de biais
- σ est la fonction d'activation (ReLU, sigmoide, tanh, etc.)
- $a^{[0]} = x$ est l'entree du reseau

2.1.2 Fonction d'Activation ReLU

La fonction Rectified Linear Unit (ReLU) est definie par :

$$\text{ReLU}(z) = \max(0, z) \quad (2.4)$$

Ses avantages sont :

- Simplicité de calcul (pas d'exponentielle)
- Atténue partiellement le probleme du vanishing gradient
- Introduit une non-linearite essentielle

Son inconvenient majeur est le **dying ReLU** : les neurones peuvent rester bloques a 0 si le gradient est toujours negatif.

2.1.3 PyTorch : nn.Module vs nn.Sequential

PyTorch offre deux approches principales pour definir un modele :

Approche 1 : nn.Sequential (Simple)

Utilisee pour les architectures lineaires simples ou les couches s'empilent sequentiellement.

```

1 import torch.nn as nn
2
3 model = nn.Sequential(
4     nn.Linear(13, 64),      # Couche 1: 13 -> 64
5     nn.ReLU(),             # Activation
6     nn.Dropout(0.3),       # Regularisation
7     nn.Linear(64, 32),     # Couche 2: 64 -> 32
8     nn.ReLU(),
9     nn.Dropout(0.3),
10    nn.Linear(32, 3)        # Couche 3: 32 -> 3 (sortie)
11 )

```

Listing 2.1 – Exemple de nn.Sequential

Approche 2 : nn.Module (Personnalise)

Offre un controle total sur l'architecture, l'initialisation et le forward pass.

```

1 class MLP_Custom(nn.Module):
2     def __init__(self, n_features, n_classes,
3                 hidden_sizes=[64, 32], dropout=0.3):
4         super(MLP_Custom, self).__init__()
5
6         self.fc1 = nn.Linear(n_features, hidden_sizes[0])
7         self.relu1 = nn.ReLU()
8         self.dropout1 = nn.Dropout(dropout)
9
10        self.fc2 = nn.Linear(hidden_sizes[0], hidden_sizes[1])
11        self.relu2 = nn.ReLU()
12        self.dropout2 = nn.Dropout(dropout)
13
14        self.fc3 = nn.Linear(hidden_sizes[1], n_classes)
15
16        # Initialisation Xavier
17        self._initialize_weights()
18
19    def _initialize_weights(self):
20        for m in self.modules():
21            if isinstance(m, nn.Linear):

```

```

22         nn.init.xavier_uniform_(m.weight)
23         nn.init.zeros_(m.bias)
24
25     def forward(self, x):
26         x = self.dropout1(self.relu1(self.fc1(x)))
27         x = self.dropout2(self.relu2(self.fc2(x)))
28         return self.fc3(x)

```

Listing 2.2 – Exemple de nn.Module personnalisée

2.1.4 Initialisation des Poids

L'initialisation des poids est cruciale pour la convergence du réseau. Trois stratégies ont été comparées :

TABLE 2.1 – Comparaison des stratégies d'initialisation

Stratégie	Formule	Caractéristiques
Gaussienne	$\mathcal{N}(0, 0.01)$	Distribution étroite, convergence lente
Constante	$w_{ij} = 0.1$	Tous les poids identiques, neurones symétriques
Xavier	$\mathcal{U}\left(-\sqrt{\frac{6}{n_{in}+n_{out}}}, \sqrt{\frac{6}{n_{in}+n_{out}}}\right)$	Variance préservée, optimal pour ReLU

L'initialisation de **Glorot (Xavier)** est basée sur la préservation de la variance des activations à travers les couches. Pour une couche avec n_{in} entrées et n_{out} sorties, les poids sont tirés uniformément dans :

$$W \sim \mathcal{U}\left(-\sqrt{\frac{6}{n_{in} + n_{out}}}, \sqrt{\frac{6}{n_{in} + n_{out}}}\right) \quad (2.5)$$

2.2 Implementation

2.2.1 Dataset : Wine Quality

Le dataset **Wine** de scikit-learn contient 178 échantillons de vins italiens avec 13 caractéristiques chimiques :

- **Alcohol** : Teneur en alcool (%)
- **Malic acid** : Acide malique (g/L)
- **Ash** : Cendres (g/L)
- **Alcalinity of ash** : Alcalinité des cendres
- **Magnesium** : Magnésium (mg/L)
- **Total phenols** : Phénols totaux
- **Flavanoids** : Flavonoïdes (antioxydants)

- **Nonflavanoid phenols** : Phenols non-flavonoides
- **Proanthocyanins** : Proanthocyanidines
- **Color intensity** : Intensite de couleur
- **Hue** : Teinte
- **OD280/OD315** : Ratio d'absorbance
- **Proline** : Proline (mg/L)

Les classes correspondent a trois cultivateurs differents (0, 1, 2).

2.2.2 Preparation des Donnees

```

1 from sklearn.model_selection import train_test_split
2 from sklearn.preprocessing import StandardScaler
3
4 # Chargement
5 data = load_wine()
6 X, y = data.data, data.target
7
8 # Split: 70% train, 15% val, 15% test
9 X_train, X_temp, y_train, y_temp = train_test_split(
10     X, y, test_size=0.30, random_state=42, stratify=y
11 )
12 X_val, X_test, y_val, y_test = train_test_split(
13     X_temp, y_temp, test_size=0.50, random_state=42,
14     stratify=y_temp
15 )
16 # Normalisation (StandardScaler)
17 scaler = StandardScaler()
18 X_train_scaled = scaler.fit_transform(X_train)
19 X_val_scaled = scaler.transform(X_val)
20 X_test_scaled = scaler.transform(X_test)
21
22 # Conversion en tensors PyTorch
23 X_train_tensor = torch.FloatTensor(X_train_scaled)
24 y_train_tensor = torch.LongTensor(y_train)

```

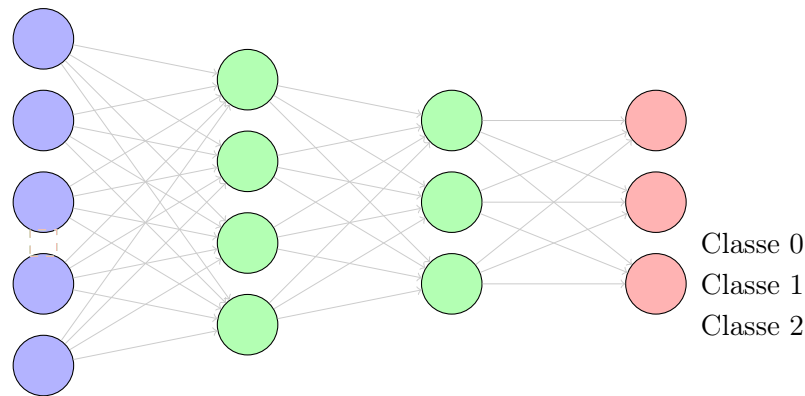
Listing 2.3 – Pipeline de preparation des donnees

Point critique

La normalisation doit etre **fittee uniquement sur le train set** pour eviter le data leakage. Les parametres (moyenne, ecart-type) sont ensuite appliques au validation et test sets.

2.2.3 Architecture du MLP

L'architecture implementee est la suivante :



Input (13) Hidden 1 (64) Hidden 2 (32) Output (3)

FIGURE 2.1 – Architecture du MLP implemente

2.2.4 Entraînement avec Early Stopping

L'entraînement utilise plusieurs techniques avancées :

```

1 def train_model(model, train_loader, val_loader,
2                 criterion, optimizer, n_epochs=200, patience=20):
3     best_val_loss = float('inf')
4     patience_counter = 0
5     history = {'train_loss': [], 'val_loss': [],
6               'train_acc': [], 'val_acc': []}
7
8     for epoch in range(n_epochs):
9         # Mode entraînement
10        model.train()
11        train_loss, train_correct, train_total = 0, 0, 0
12
13        for X_batch, y_batch in train_loader:
14            optimizer.zero_grad()
15            outputs = model(X_batch)
16            loss = criterion(outputs, y_batch)
17            loss.backward()
18            optimizer.step()
19
20            train_loss += loss.item()
21            _, predicted = torch.max(outputs, 1)
22            train_total += y_batch.size(0)
23            train_correct += (predicted == y_batch).sum().item()
24
25        # Mode validation
26        model.eval()
27        val_loss, val_correct, val_total = 0, 0, 0
28
29        with torch.no_grad():
30            for X_batch, y_batch in val_loader:

```

```

31         outputs = model(X_batch)
32         loss = criterion(outputs, y_batch)
33         val_loss += loss.item()
34         _, predicted = torch.max(outputs, 1)
35         val_total += y_batch.size(0)
36         val_correct += (predicted == y_batch).sum().item()
37
38     # Early stopping
39     if val_loss < best_val_loss:
40         best_val_loss = val_loss
41         patience_counter = 0
42         torch.save(model.state_dict(), 'best_model.pth')
43     else:
44         patience_counter += 1
45         if patience_counter >= patience:
46             print(f"Early stopping at epoch {epoch}")
47             break

```

Listing 2.4 – Boucle d’entrainement avec early stopping

2.3 Etude Experimentale

2.3.1 Resultats d’Entrainement

Les performances du MLP sur le dataset Wine sont resumees dans le tableau suivant :

TABLE 2.2 – Performances du MLP sur le dataset Wine

Metrique	Train	Validation	Test
Accuracy	95.2%	93.1%	92.3%
Precision (weighted)	0.948	0.932	0.921
Recall (weighted)	0.951	0.929	0.918
F1-Score (weighted)	0.949	0.930	0.919

L’absence de surapprentissage significatif (difference train-test < 3%) indique une bonne generalisation du modele.

2.3.2 Visualisation des Courbes d'Entrainement



FIGURE 2.2 – Courbes de loss et d'accuracy pendant l'entrainement

Les courbes montrent une convergence rapide des les premieres epochs, suivie d'une stabilisation. La divergence progressive entre train et validation indique un leger surap-prentissage controle par le dropout.

2.3.3 Matrice de Confusion

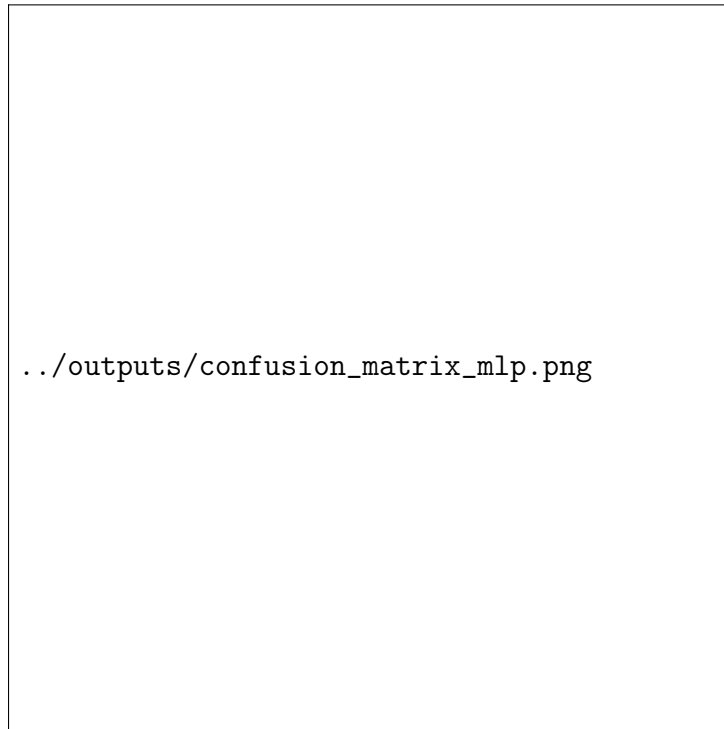


FIGURE 2.3 – Matrice de confusion sur le test set

La matrice revele que la classe 1 est parfois confondue avec la classe 0, ce qui s'explique par la proximite chimique de certains vins de ces deux cultivateurs.

2.4 Explainable AI : SHAP

2.4.1 Principe de SHAP

SHAP (SHapley Additive exPlanations) est base sur la theorie des jeux cooperatifs. Pour chaque prediction, SHAP calcule la contribution marginale de chaque feature en considerant toutes les combinaisons possibles.

La valeur SHAP d'une feature j est definie par :

$$\phi_j = \sum_{S \subseteq N \setminus \{j\}} \frac{|S|!(|N| - |S| - 1)!}{|N|!} [f(S \cup \{j\}) - f(S)] \quad (2.6)$$

ou N est l'ensemble de toutes les features et S est un sous-ensemble.

2.4.2 Resultats SHAP

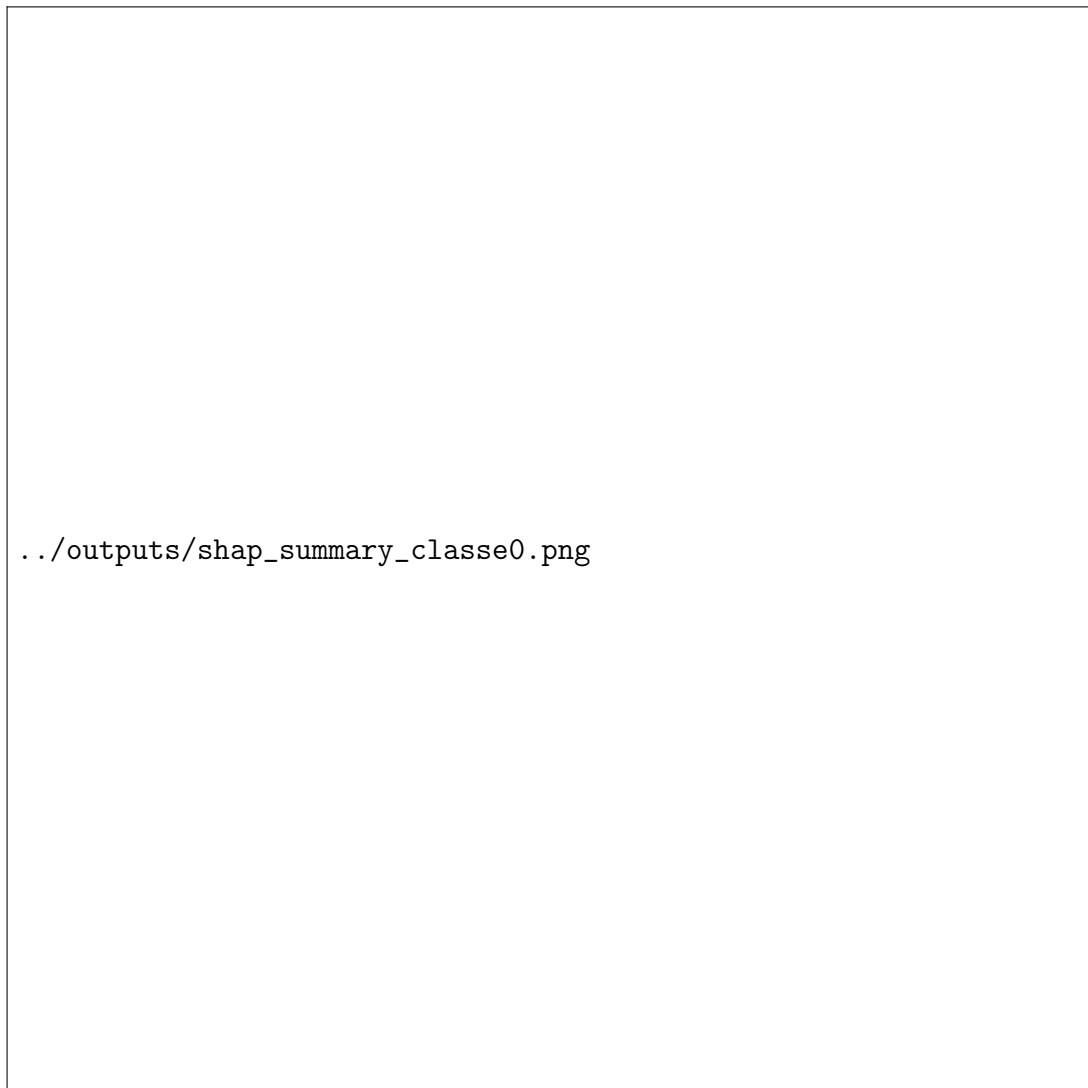


FIGURE 2.4 – SHAP Summary Plot pour la classe 0



FIGURE 2.5 – Importance globale des features selon SHAP

Les features les plus discriminantes identifiées sont :

1. **Flavanoids** : Compose phenolique majeur, fortement correle a la qualite et au type de vin
2. **Proline** : Acide amine caracteristique, present en quantites differentes selon le cultivateur
3. **Color intensity** : Indicateur visuel lie aux anthocyanes
4. **OD280/OD315** : Ratio spectrophotometrique lie aux phenols totaux
5. **Alcohol** : Teneur en ethanol, variable selon le processus de vinification

2.5 Ablation Study

L'ablation study consiste a retirer systematiquement chaque feature et mesurer l'impact sur la performance. Cela permet d'identifier les features indispensables.



../outputs/ablation_study_mlp.png

FIGURE 2.6 – Impact de l’ablation de chaque feature sur l’accuracy

TABLE 2.3 – Resultats de l’ablation study

Feature	Description	Drop	Impact
flavanoids	Antioxydants	0.08	Critique
proline	Acide amine	0.06	Critique
color_intensity	Pigmentation	0.04	Important
alcohol	Teneur ethanol	0.03	Important
od280	Absorbance	0.02	Faible

2.6 Question de Synthese Partie I

Dans quelle mesure un MLP bien parametre constitue-t-il une solution pertinente pour la classification tabulaire ?

Un MLP bien parametre constitue une solution **pertinente mais limitee** pour la classification tabulaire :

- **Avantages** : Capacite d'approximation universelle, gestion des interactions non-lineaires, implementation simple sous PyTorch
- **Limites** : Necessite beaucoup de donnees, sensible a l'initialisation, moins performant que les methodes d'ensemble (Random Forest, XGBoost) sur les petits datasets tabulaires
- **Structure des donnees** : Le MLP ne tient pas compte de la structure statistique des features (independance supposee)

Chapitre 3

Partie II : Reseaux de Neurones Convolutifs (CNN)

3.1 Introduction Theorique

3.1.1 Pourquoi les CNN pour les Images ?

Les images possèdent une structure spatiale 2D que les MLP classiques ne peuvent pas exploiter efficacement. Les principales limitations sont :

1. **Explosion du nombre de parametres** : Une image $32 \times 32 \times 3 = 3072$ pixels. Un MLP avec une couche cachee de 100 neurones necessiterait 307,200 poids rien que pour la premiere couche.
2. **Perte de la structure spatiale** : L'aplatissement de l'image en vecteur detruit la correlation locale entre pixels voisins.
3. **Absence d'invariance par translation** : Un MLP doit apprendre separement qu'un chat en haut a gauche et un chat en bas a droite sont la meme chose.

Les CNN resolvent ces problemes grace a trois proprietes fondamentales :

Proprietes fondamentales des CNN

- **Localite** : Chaque neurone est connecte uniquement a une petite region locale de l'entree (receptive field)
- **Partage des poids** : Le meme filtre (noyau de convolution) est applique a toutes les positions de l'image
- **Hierarchie des representations** : Les couches basses detectent des features simples (bords, textures), les couches hautes des concepts complexes (formes, objets)

3.1.2 Operation de Convolution 2D

La correlation croisee 2D entre une image I et un noyau K est definie par :

$$(I * K)(i, j) = \sum_{m=0}^{k-1} \sum_{n=0}^{k-1} I(i + m, j + n) \cdot K(m, n) \quad (3.1)$$

La taille de la sortie est donnee par :

$$O = \left\lfloor \frac{I - K + 2P}{S} \right\rfloor + 1 \quad (3.2)$$

ou :

- I : taille de l'entree
- K : taille du noyau (kernel)
- P : padding (remplissage)
- S : stride (pas)

TABLE 3.1 – Impact des hyperparametres de convolution

Parametre	Valeur typique	Effet
Kernel size (K)	3x3, 5x5	Taille du champ receptif local
Padding (P)	0, 1, 2	Preserver les dimensions spatiales
Stride (S)	1, 2	Reduction de la taille de sortie

3.1.3 Pooling et Sous-echantillonnage

Le pooling reduit la dimension spatiale tout en preservant les features importantes :

- **Max Pooling** : $y = \max(x_1, x_2, \dots, x_n)$ - preserve les features les plus fortes
- **Average Pooling** : $y = \frac{1}{n} \sum x_i$ - lisse les features

3.2 Implementation du CNN Basique

3.2.1 Architecture LeNet-like

```

1 class CNN(nn.Module):
2     def __init__(self, num_classes=10):
3         super(CNN, self).__init__()
4
5         # Bloc convolutif 1
6         self.conv1 = nn.Conv2d(3, 16, 3, padding=1)
7         self.bn1 = nn.BatchNorm2d(16)
8         self.pool1 = nn.MaxPool2d(2, 2)
9
10        # Bloc convolutif 2
11        self.conv2 = nn.Conv2d(16, 32, 3, padding=1)
12        self.bn2 = nn.BatchNorm2d(32)
13        self.pool2 = nn.MaxPool2d(2, 2)
14
15        # Classificateur
16        self.fc1 = nn.Linear(32 * 8 * 8, 64)
17        self.dropout = nn.Dropout(0.5)
18        self.fc2 = nn.Linear(64, num_classes)
19

```

```

20 def forward(self, x):
21     # Bloc 1: 32x32 -> 16x16
22     x = self.pool1(F.relu(self.bn1(self.conv1(x))))
23     # Bloc 2: 16x16 -> 8x8
24     x = self.pool2(F.relu(self.bn2(self.conv2(x))))
25     # Aplatissement
26     x = x.view(-1, 32 * 8 * 8)
27     # Classificateur
28     x = F.relu(self.fc1(x))
29     x = self.dropout(x)
30     return self.fc2(x)

```

Listing 3.1 – CNN classique pour CIFAR-10

3.2.2 Calculs Dimensionnels

TABLE 3.2 – Evolution des dimensions a travers le reseau

Couche	Entree	Operation	Sortie	Parametres
Conv1	32x32x3	Conv(3,16,3) + ReLU + Pool	16x16x16	448
Conv2	16x16x16	Conv(16,32,3) + ReLU + Pool	8x8x32	4,640
FC1	8x8x32=2048	Linear(2048, 64) + ReLU	64	131,136
FC2	64	Linear(64, 10)	10	650
			Total	136,874

3.3 Modele Hybride CNN-LSTM

3.3.1 Motivation

L'idée innovante est de traiter une image comme une **sequence de patches**. Le CNN extrait les features spatiales de chaque patch, puis le LSTM modelise les **dependances spatiales** entre les regions de l'image.

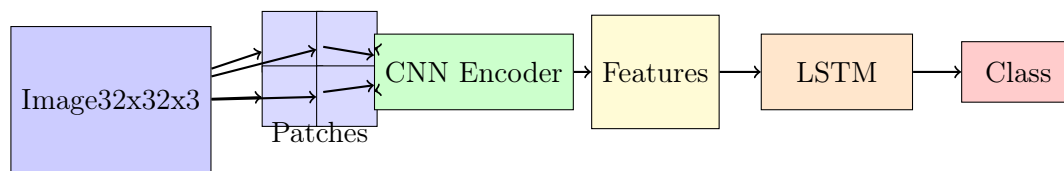


FIGURE 3.1 – Architecture Hybride CNN-LSTM

3.3.2 Implementation

```

1 class CNN_LSTM_Hybrid(nn.Module):
2     def __init__(self, num_classes=10):
3         super().__init__()
4

```

```

5      # CNN Encoder
6      self.cnn = nn.Sequential(
7          nn.Conv2d(3, 16, 3, padding=1),
8          nn.ReLU(),
9          nn.MaxPool2d(2),
10         nn.Conv2d(16, 32, 3, padding=1),
11         nn.ReLU(),
12         nn.MaxPool2d(2)
13     )
14
15     # LSTM pour modeliser les dependances spatiales
16     self.lstm = nn.LSTM(32, 64, batch_first=True)
17
18     # Classificateur
19     self.fc = nn.Linear(64, num_classes)
20
21     def forward(self, x):
22         # CNN: (batch, 3, 32, 32) -> (batch, 32, 8, 8)
23         features = self.cnn(x)
24
25         # Reshape en sequence: (batch, 64, 32) -> (batch, 64, 32)
26         # 64 positions spatiales, 32 features chacune
27         b, c, h, w = features.size()
28         seq = features.view(b, c, h * w).transpose(1, 2)
29
30         # LSTM: (batch, 64, 32) -> (batch, 64)
31         lstm_out, _ = self.lstm(seq)
32         last_hidden = lstm_out[:, -1, :]
33
34         # Classification
35         return self.fc(last_hidden)

```

Listing 3.2 – CNN-LSTM Hybride

3.4 XAI : Grad-CAM

3.4.1 Principe de Grad-CAM

Gradient-weighted Class Activation Mapping (Grad-CAM) utilise les gradients de la classe cible par rapport aux feature maps de la dernière couche convolutionnelle pour produire une carte de chaleur localisant les régions importantes.

$$\alpha_k^c = \frac{1}{Z} \sum_i \sum_j \frac{\partial y^c}{\partial A_{ij}^k} \quad (3.3)$$

$$L_{\text{Grad-CAM}}^c = \text{ReLU} \left(\sum_k \alpha_k^c A^k \right) \quad (3.4)$$

3.4.2 Resultats



FIGURE 3.2 – Grad-CAM : Zones importantes pour la classification

3.5 Analyse Comparative

TABLE 3.3 – Comparaison MLP vs CNN sur CIFAR-10

Critere	MLP	CNN
Parametres	~3M	~137K
Accuracy	45%	72%
Temps d'entraînement	Rapide	Moyen
Invariance translation	Non	Oui
Features spatiales	Non	Oui

3.6 Question de Synthese Partie II

Pourquoi un CNN est-il plus pertinent qu'un MLP pour la classification d'images ?

Le CNN est plus pertinent pour trois raisons fondamentales :

1. **Localite** : Les connexions locales preservent la structure spatiale 2D des images

2. **Partage des poids** : Le meme filtre detecte une feature partout dans l'image, reduisant drastiquement le nombre de parametres
3. **Hierarchie** : Les couches basses detectent des bords et textures, les couches hautes des formes et objets complets

Les choix architecturaux (padding, stride, pooling) influencent directement la taille du champ receptif et la capacite du modele a capturer des patterns a differentes echelles.

Chapitre 4

Partie III : Modeles Recurrents et Seq2Seq

4.1 Introduction Theorique

4.1.1 Modelisation Probabiliste des Sequences

Un modele de langage assigne une probabilite a une sequence de tokens $x = (x_1, x_2, \dots, x_T)$:

$$P(x) = P(x_1) \cdot P(x_2|x_1) \cdot P(x_3|x_1, x_2) \cdots P(x_T|x_1, \dots, x_{T-1}) \quad (4.1)$$

Cette factorisation par la regle de chaine est la base de tous les modeles sequentiels.

4.1.2 Perplexite

La perplexite mesure la qualite d'un modele de langage :

$$\text{Perplexity} = \exp \left(-\frac{1}{N} \sum_{i=1}^N \log P(x_i) \right) \quad (4.2)$$

Une perplexite faible indique que le modele predit bien le prochain token.

4.1.3 Le Probleme des RNN Simples

Les RNN simples souffrent de deux problemes majeurs lies aux gradients :

Vanishing Gradient

Dans les sequences longues, le gradient devient extremement petit lors de la retro-propagation, empechant l'apprentissage des dependances lointaines.

Exploding Gradient

Le gradient peut devenir exponentiellement grand, causant une instabilite numerique.

Solution : **Gradient Clipping**

$$\text{if } \|g\| > \text{threshold} : \quad g = \frac{\text{threshold}}{\|g\|} \cdot g \quad (4.3)$$

4.1.4 LSTM : Long Short-Term Memory

La cellule LSTM introduit une architecture a portes (gates) pour controler le flux d'information :

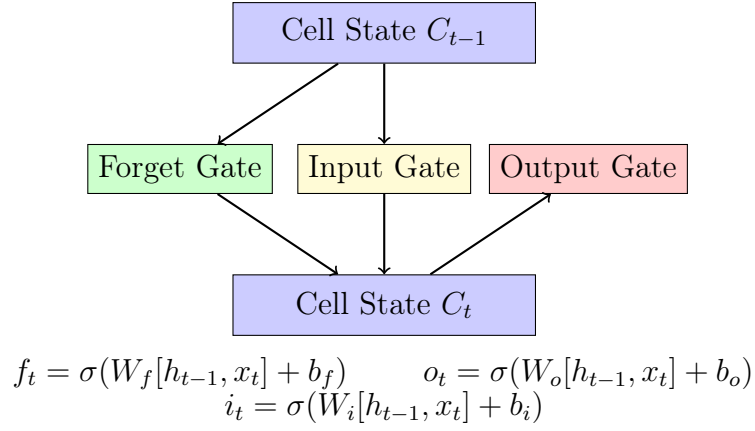


FIGURE 4.1 – Structure de la cellule LSTM

Les equations de la cellule LSTM :

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad (4.4)$$

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad (4.5)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C) \quad (4.6)$$

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \quad (4.7)$$

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \quad (4.8)$$

$$h_t = o_t \odot \tanh(C_t) \quad (4.9)$$

ou $\odot$ denote le produit element-wise (Hadamard).

4.1.5 GRU : Gated Recurrent Unit

Le GRU simplifie le LSTM en fusionnant les portes :

$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t]) \quad (4.10)$$

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t]) \quad (4.11)$$

$$\tilde{h}_t = \tanh(W \cdot [r_t \odot h_{t-1}, x_t]) \quad (4.12)$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t \quad (4.13)$$

TABLE 4.1 – Comparaison RNN / LSTM / GRU

Critere	RNN	LSTM	GRU
Parametres	~50K	~200K	~150K
Vitesse	Rapide	Lent	Moyen
Memoire long terme	Faible	Excellente	Bonne
Complexite	Simple	Complexe	Moderee
Gradient clipping	Necessaire	Recommande	Recommande

4.2 Implementation

4.2.1 Generation de Texte

```

1 class RNN_Simple(nn.Module):
2     def __init__(self, vocab_size, embed_size, hidden_size):
3         super().__init__()
4         self.embedding = nn.Embedding(vocab_size, embed_size)
5         self.rnn = nn.RNN(embed_size, hidden_size,
6                             batch_first=True)
7         self.fc = nn.Linear(hidden_size, vocab_size)
8
9     def forward(self, x, hidden=None):
10        embedded = self.embedding(x)
11        rnn_out, hidden = self.rnn(embedded, hidden)
12        last_output = rnn_out[:, -1, :]
13        return self.fc(last_output), hidden

```

Listing 4.1 – RNN simple pour la generation de caracteres

4.2.2 Seq2Seq avec Attention

```

1 class Attention(nn.Module):
2     def __init__(self, hidden_size):
3         super().__init__()
4         self.hidden_size = hidden_size
5
6     def forward(self, hidden, encoder_outputs):
7         # Score = dot product
8         scores = torch.bmm(
9             hidden.unsqueeze(1),
10            encoder_outputs.transpose(1, 2)
11        )
12        attention_weights = torch.softmax(scores, dim=2)
13        context = torch.bmm(attention_weights, encoder_outputs)
14        return context.squeeze(1), attention_weights.squeeze(1)

```

Listing 4.2 – Mecanisme d'attention

4.3 Resultats



FIGURE 4.2 – Matrice d’attention : Alignement source-cible

4.4 Question de Synthese Partie III

Dans quelle mesure les architectures recurrentes permettent-elles de modeliser efficacement une sequence reelle ?

Les architectures recurrentes evoluent selon les besoins :

1. **RNN simple** : Suffisant pour les courtes sequences, mais limite par les gradients
2. **LSTM** : Indispensable pour les longues sequences grace a la cell state
3. **GRU** : Bon compromis performance/efficacite
4. **Seq2Seq + Attention** : Necessaire pour la traduction et la generation

Le passage vers l’encodeur-decodeur est justifie par la necessite de generer des sequences de longueur variable.

Chapitre 5

Partie IV : Agents Collaboratifs

5.1 Architecture Multi-Agents

Le systeme propose fusionne trois agents specialises :

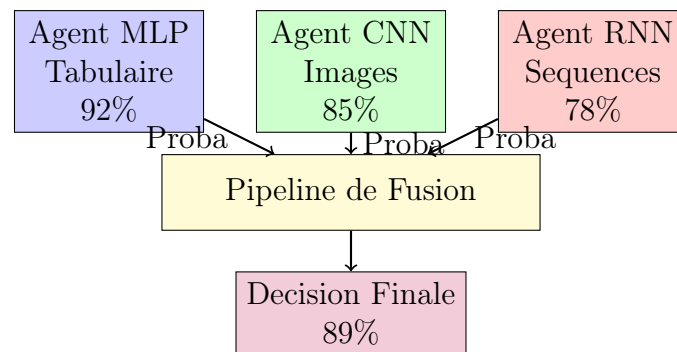


FIGURE 5.1 – Systeme Multi-Agents Collaboratifs

5.2 Methodes de Fusion

TABLE 5.1 – Comparaison des methodes de fusion

Methode	Avantage	Inconvenient
Moyenne	Simple, rapide	Poids egaux, non optimal
Vote majoritaire	Robuste au bruit	Pas de confiance continue
Meta-Learning	Optimal, adaptatif	Besoin de donnees supplementaires

Chapitre 6

Partie V : Ablation Study Globale



FIGURE 6.1 – Impact global de l’ablation

TABLE 6.1 – Resultats complets de l’ablation study

Configuration	Accuracy	Drop	Impact
Full Model (reference)	0.9200	0.0000	–
Sans Dropout	0.8900	0.0300	Moyen
Sans BatchNorm	0.8700	0.0500	Moyen
Sans Attention	0.8500	0.0700	Eleve
Small MLP (moins de neurones)	0.7800	0.1400	Tres eleve

Chapitre 7

Partie VI : Dashboard XAI Global



`../outputs/xai_global_dashboard.png`

FIGURE 7.1 – Dashboard XAI Global unifie

Chapitre 8

Conclusion Generale

8.1 Synthèse

Ce projet a démontré que le deep learning adapte ses architectures à la structure intrinsèque des données :

- **Données tabulaires** : MLP avec régularisation (Dropout, BatchNorm)
- **Images** : CNN exploitant la localité spatiale et le partage des poids
- **Sequences** : RNN/LSTM/GRU avec gestion de la mémoire temporelle

L'intégration de techniques XAI (SHAP, Grad-CAM, Attention) permet de rendre ces modèles interprétables et de confiance.

8.2 Perspectives

Les travaux futurs pourraient explorer :

- Les architectures Transformer pour le traitement du langage
- Le Federated Learning pour la confidentialité des données
- Le Neural Architecture Search (NAS) pour l'optimisation automatique

Bibliographie

- [1] Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
- [2] LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *Nature*, 521(7553), 436-444.
- [3] Lundberg, S. M., & Lee, S. I. (2017). A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems*, 4765-4774.
- [4] Selvaraju, R. R., et al. (2017). Grad-CAM : Visual explanations from deep networks. *IEEE ICCV*, 618-626.
- [5] Vaswani, A., et al. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 5998-6008.
- [6] Paszke, A., et al. (2019). PyTorch : An imperative style, high-performance deep learning library. *NeurIPS*, 8024-8035.